



Por dentro do mundo do DynamoDB

A Quest da velocidade



Nossa Jornada de Hoje

3 horas total

01

10 min

 ABERTURA

02

45 min

Quest 1: FUNDAMENTOS

-  Teoria: SQL vs NoSQL
-  Teoria: Conceitos DynamoDB
-  Arena: Criar Tabela + Operações
-  Quiz 1: Fundamentos do Dynamo

03

55 min

Quest 2: MODELAGEM

-  Vídeo: Família Real
-  Teoria: Single-Table Design
-  Prática: Exercício de modelagem guiado
-  Arena: Queries básicas
-  Quiz 2: Access Patterns + Queries

04

10 min

 PAUSA

05

45 min

Quest 3: GSIs

-  Teoria: GSI
-  Arena: Operações com GSI
-  Quiz 3: GSI

06

15 min

Quest 4: FECHAMENTO

-  Boas Práticas
-  Ranking & Feedback



Bancos não relacionais

Fundamentos do DynamoDB



O Que É Um Banco Não Relacional?

Os bancos de dados não relacionais, ou NoSQL (Not Only SQL), representam uma abordagem diferente para o armazenamento e recuperação de dados em comparação com os bancos de dados relacionais tradicionais. Eles são projetados para lidar com grandes volumes de dados não estruturados em alta velocidade, oferecendo flexibilidade e escalabilidade para necessidades modernas de aplicações.

Schema Flexível

Cada item pode ter atributos diferentes, permitindo evoluir o modelo de dados facilmente sem a necessidade de definir um esquema rígido antecipadamente.

Sem JOINS

Os dados já vêm "prontos" e agrupados, otimizados para padrões de acesso específicos, eliminando a complexidade e o custo de operações JOIN.

Escalabilidade Horizontal

Permitem o crescimento ao distribuir dados entre múltiplos servidores (sharding), facilitando o gerenciamento de volumes massivos de informações e alta demanda de tráfego.

Alto Desempenho

São otimizados para leitura e escrita em alta velocidade, ideal para aplicações que exigem baixa latência e processamento rápido de transações.

Múltiplos Formatos

Suportam diversos modelos de dados, incluindo documentos (JSON), chave-valor, colunas e gráficos, adaptando-se melhor a diferentes tipos de dados e casos de uso.

Implementação Rápida

Sua natureza flexível permite o registro e a análise de novas informações rapidamente, acelerando o ciclo de desenvolvimento e a inovação.

SQL vs NoSQL - Comparação

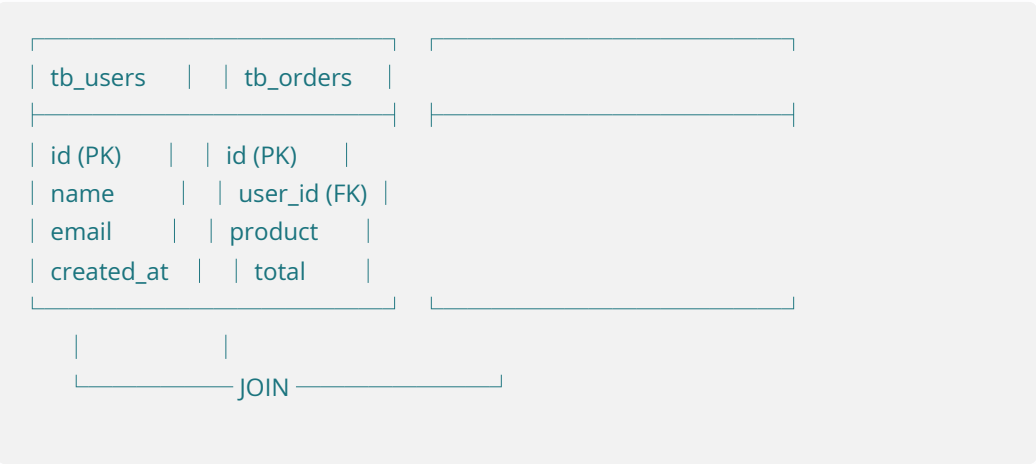
Para entender melhor as vantagens de um banco de dados não relacional como o DynamoDB, vamos comparar suas características com as dos bancos de dados relacionais tradicionais.

SQL (Relacional)	NoSQL (Não Relacional)
 Múltiplas Tabelas	 Single Table (ou poucas tabelas)
 JOINS Necessários para relacionar	 Sem JOINS. Dados já juntos
 Schema Rígido. Colunas fixas definidas antes	 Schema Flexível. Atributos variam por item
 Performance Variável (depende de JOINS)	 Performance Previsível (ms)
 Normalização. Dados separados em tabelas	 Denormalização. Dados duplicados estrategicamente

Exemplo de Schema

Veja como a organização dos dados muda de um modelo relacional para um não relacional, eliminando a necessidade de JOINS.

SQL (Relacional)



NoSQL (DynamoDB)



Tudo junto, sem JOINS!



O Que é DynamoDB?

O Amazon DynamoDB é um serviço de banco de dados NoSQL totalmente gerenciado, serverless e distribuído, oferecido pela AWS. Projetado para aplicações que exigem alta performance, escalabilidade e disponibilidade, sem a complexidade de gerenciar infraestrutura.

Serverless e Totalmente Gerenciado

Zero infraestrutura para gerenciar, zero downtime, zero janelas de manutenção. A AWS cuida de backups, patches e segurança.

Performance em Milissegundos

Latência consistente de single-digit milliseconds (1-10ms) em qualquer escala, desde 10 até 100 milhões de usuários.

Escalabilidade Automática

Ajusta instantaneamente a capacidade de leitura e escrita conforme a demanda da sua aplicação, sem intervenção manual.

Alta Disponibilidade

Garante 99.99% a 99.999% de SLA com replicação automática em múltiplas zonas de disponibilidade.

Schema Flexível

Modelo de dados flexível que se adapta às mudanças de requisitos sem necessidade de redefinir a tabela.

Distribuído em Partições

Dados automaticamente divididos e replicados em múltiplas zonas para resiliência e acesso rápido.



Estrutura do DynamoDB



Modelo de Dados

- Tabela: conjunto de itens.
- Partition Key (PK): identifica o grupo de itens.
- Sort Key (SK): organiza itens dentro da mesma PK.



Partições

- Partição lógica: todos os itens com a mesma PK.
- Partições físicas: blocos internos de armazenamento.
- DynamoDB usa o hash da PK para distribuir itens entre partições físicas.



Hot Partition e Escalabilidade

- Hot partition: muitas operações concentradas na mesma PK.
- DynamoDB cria/ajusta partições físicas automaticamente para escalar.

Operações Básicas do DynamoDB

O DynamoDB oferece um conjunto robusto de operações para manipulação de dados, divididas entre leitura e escrita. Compreender suas diferenças é chave para otimizar a performance e custo de suas aplicações.

Operações de Leitura

→ GetItem

Busca **1 item específico** usando a Partition Key (e Sort Key, se composta). Oferece tempo de resposta de **ate 3 ms**. Ideal para recuperação direta de um único registro.

→ Query

Recupera múltiplos itens que compartilham a **mesma Partition Key**. É eficiente, tempo de resposta de **ate 8ms**, permite filtros na Sort Key e paginates automaticamente (1MB por operação).

→ Scan

Varre a **tabela inteira** sequencialmente, retornando todos os itens. A latência é **variável** e depende do tamanho da tabela. Deve ser **evitado em produção** devido ao alto custo e ineficiência.

Operações de Escrita

→ PutItem

Cria um novo item ou **substitui completamente** um item existente pela Partition Key (e Sort Key).. Limite de 400 KB por item.

→ UpdateItem

Atualiza **atributos específicos** de um item existente. Isso evita a necessidade de reescrever o item inteiro, sendo mais eficiente.

→ DeleteItem

Remove um item da tabela. Requer a Partition Key (e Sort Key, se composta) para identificar o item.

→ BatchWriteItem

Permite realizar **múltiplas operações** (PutItem ou DeleteItem) em lote. Útil para eficiência em grandes volumes, com limite de 25 itens ou 16 MB por chamada.



Operações na Sort Key (SK)

A Sort Key (SK) permite refinar consultas dentro de uma partição. Você pode usar diferentes operadores para buscar ranges de dados, prefixos específicos e muito mais. Dominar essas operações é essencial para otimizar suas queries.

Igualdade (=)

- Busca: Um item exato
- Exemplo: SK = "SUMMARY"
- Resultado: 1 item específico

Começa Com (begins_with)

- Busca: Todos os itens com SK iniciando com um prefixo
- Exemplo: begins_with(SK, "PLAN#")
- Resultado: Múltiplos itens ordenados

Intervalo (BETWEEN)

- Busca: Itens dentro de um range de valores
- Exemplo: SK BETWEEN "2024-01-01" AND "2024-12-31"
- Resultado: Múltiplos itens ordenados

Comparações (>, <, >=, <=)

- Busca: Itens maiores, menores ou iguais a um valor
- Exemplo: SK >= "2024-01-01"
- Resultado: Múltiplos itens ordenados

Limitações Importantes

- PK SEMPRE usa igualdade (=)
- begins_with, BETWEEN e comparações funcionam APENAS na SK
- contains() NÃO funciona em KeyConditionExpression (use FilterExpression para isso)



Casos de Uso Comuns

- Buscar todos os planos de um funcionário
 - Usar: PK (identificador do funcionário) + begins_with(SK, "PLAN#")
 - Resultado: Lista de todos os planos associados àquele funcionário
- Buscar saldos em um período específico
 - Usar: PK (identificador do usuário) + BETWEEN (datas de início e fim)
 - Resultado: Todos os registros de saldos entre as duas datas especificadas
- Buscar histórico a partir de uma data
 - Usar: PK (identificador do usuário) + >= (data de início)
 - Resultado: Todos os registros históricos a partir da data informada



Hora da Prática: Sua Primeira Missão!

Agora que você entende os fundamentos do DynamoDB, é hora de colocar a mão na massa. Siga as 3 fases para dominar as operações essenciais.

FASE 1: Criação de Tabelas

Objetivo: Criar a estrutura base para suas operações.

- **Quest 1.1: Tabela simples** (lab1-chave-simples)
PK: PK (String)
- **Quest 1.2: Tabela composta** (lab1-chave-composta)
PK: PK (String), SK: SK (String)

Verificação: Listar tabelas criadas.

FASE 2: Operações de Escrita

Objetivo: Inserir, atualizar e deletar dados em suas tabelas.

- **Quest 2.1: Inserção Simples** (PutItem)
- **Quest 2.2: Atualização** (UpdateItem)
- **Quest 2.3: Deleção** (DeleteItem)
- **Quest 2.4: Inserção em Lote** (BatchWriteItem)

FASE 3: Operações de Leitura

Objetivo: Recuperar dados de diferentes formas.

- **Quest 3.1: GetItem** (recuperar item específico)
- **Quest 3.2: Query com PK** (todos da partição)
- **Quest 3.3: Query com PK + SK** (range de SK)
- **Quest 3.4: Scan** (varrer tabela - use com cuidado!)



Você está aqui: Quiz 1

Quest 1

Onde estamos:

- ✓ Conceitos do DynamoDB
- ✓ SQL vs NoSQL

▶ AGORA: Quiz 1 - Conceitos Básicos (5min)

- └ Validar conhecimento teórico

O que vem depois:

- ➡ Quest 2: Modelagem (teoria + prática)

Quest 1 quase completo! ✨



Você está aqui: Vídeo: Família Real

Quest 2 de 4

Onde estamos:

-  Quest 1: Fundamentos (completo!)

 AGORA: Vídeo: Família real (5min)

Próximos passos:

-  Teoria: Single-Table Design
-  Prática: Exercício de modelagem guiado
-  Arena: Queries básicas
-  Quiz 2: Access Patterns + Queries

Você está aqui: Modelagem Teórica

Quest 2 de 4

Onde estamos:

-  Quest 1: Fundamentos (completo!)

 AGORA:
Modelagem Teórica
(10min)

Próximos passos:

-  Prática: Exercício de modelagem guiado
-  Arena: Queries básicas
-  Quiz 2: Access Patterns + Queries





Quest DynamoDB: Modelagem de Dados

Do Conhecimento das Entidades à Estrutura Final

Pense diferente. Modele para velocidade.



Sua Jornada de Conhecimento de Access Patterns

Nesta Academia 🏛️, você vai dominar os conceitos essenciais para modelar dados no DynamoDB com eficiência e Performance ⚡.

Vamos explorar como pensar diferente do SQL tradicional e aplicar técnicas que aproveitam ao máximo as capacidades do DynamoDB, garantindo que você esteja preparado para qualquer Missão 🗡️!

01

Mudança de mentalidade

Defina como os dados serão acessados primeiro

02

Access Pattern: A Pergunta que Define Tudo

Como criar padrões de acesso

03

Usar Prefixos como Namespace

Organize e isole diferentes tipos de dados dentro de uma única tabela.

04

Aplicar Padrão SUMMARY

Resuma dados relacionados de forma eficiente para acesso rápido.

05

Usar Séries Temporais eficientemente

Gerencie dados baseados em tempo com alta performance.

06

Aplicar Denormalização Estratégica

Saiba quando e como duplicar dados para eficiência.

07

Filter Expressions

Refinando resultados da query

👑 **Objetivo Final:** Chegar na estrutura completa e otimizada do banco de dados, tornando-se um Mestre 👑 em DynamoDB e ganhando um 💎 Bônus de conhecimento!

1 Mudança de Mentalidade: Access Pattern First

Adquira esta **Habilidade Essencial** para sua **Quest DynamoDB!**

SQL Tradicional

1. Começa pelas **ENTIDADES**
2. Normaliza em 3FN
3. Cria foreign keys
4. Depois faz **JOINS** nas queries (potenciais gargalos 🐢)

DynamoDB

1. Começa pelos **ACCESS PATTERNS**
2. Modela para esses padrões
3. Zero JOINS necessários
4. Performance constante

🌟 **Insight chave:** "Como vou **USAR** os dados?" define a modelagem. Isso é crucial para o sucesso da sua missão! 🗡️

❌ Abordagem Incorreta

"Tenho USER, EMPLOYEE, PLAN..."

✅ Abordagem Correta

"Preciso buscar dados do employee rapidamente"

"Preciso listar todos os planos de um employee"

💡 Aha moment:

- No SQL você modela os DADOS e depois descobre como acessá-los.
- No DynamoDB você modela o ACESSO e depois encaixa os dados.

🏛️ Vale pensar:

Na modelagem de uma tabela não relacional não existe um único formato correto, modelamos tentando deixar os acessos que mais teremos que fazer o mais simples possíveis, mas não existe uma solução apenas.



2 Access Pattern: A Pergunta que Define Tudo

No DynamoDB, a pergunta fundamental não é "Quais dados eu tenho?", mas sim: "**Como minha aplicação vai USAR esses dados?**"

Um **Access Pattern** é a descrição de **UMA** operação que sua aplicação precisa realizar, focando na forma como você **ACESSA** os dados.

Formato Padrão

[AÇÃO] + [O QUÊ] + [CONDIÇÃO]

Exemplos de Access Patterns

- Buscar dados de um employee pelo UUID
- Listar todos os planos de um employee
- Buscar o último saldo de um certificado
- Listar certificados de um plano
- Buscar transações de um certificado em um período
- Verificar credenciais de login de um usuário

Princípios Fundamentais

LISTE ANTES DE MODELAR

Antes de criar qualquer tabela, liste **TODOS** os access patterns. Pergunte: "Quais telas existem? Quais dados cada uma precisa?"

PRIORIZE POR FREQUÊNCIA

Access patterns mais usados = modelagem otimizada para eles. Se um login acontece 1000x/dia, ele é prioridade máxima!

IDENTIFIQUE O "QUEM"

Todo access pattern tem um ponto de entrada. "Buscar planos DE UM EMPLOYEE" → Employee é o "quem" → vira PK!

IDENTIFIQUE O "O QUÊ"

O que você quer buscar a partir do "quem"? "Buscar PLANOS de um employee" → Planos é o "o quê" → vira SK!

AGRUPE PADRÕES SIMILARES

Patterns com o mesmo "quem" podem compartilhar a mesma PK. Ex: "Dados do employee" + "Planos do employee" = mesma PK, SKs diferentes!

3 PREFIXOS COMO NAMESPACE

📄 Convenção:

Use **prefixos** claros em PK e SK para criar "namespaces" lógicos. Isso organiza seus dados como uma biblioteca bem indexada! 📖

Exemplos na nossa estrutura:

USER#	USER#550e8400	Identifica usuários
EMPLOYEE#	EMPLOYEE#650e8400	Identifica colaboradores
PLAN#	PLAN#770e8400	Identifica planos
CERTIFICATE#	CERTIFICATE#aa0e8400	Identifica certificados
BALANCE#	BALANCE#2024-11-15	Identifica saldos (com data)
TRANSACTION#	TRANSACTION#2024-11-05#143000	Identifica transações (com timestamp)

🌐 Benefícios de usar prefixos:

- **Código auto-documentado:** A estrutura do seu código fica mais legível e intuitiva, facilitando a compreensão para qualquer desenvolvedor.
- **Evita colisões de UUIDs:** Mesmo que diferentes tipos de entidade usem UUIDs similares, os prefixos garantem que eles sejam únicos dentro da mesma tabela.
- **Facilita `begins_with` queries:** Agrupe e consulte eficientemente itens que começam com um prefixo específico, ideal para buscar todos os itens de um tipo.
- **Debug mais fácil:** Identificar e depurar problemas se torna mais rápido, pois você pode facilmente ver qual tipo de dado está associado a cada PK/SK.

4 Padrão SUMMARY: Convenção para Dados Principais

Adotamos a convenção de usar SK = "SUMMARY" para armazenar os dados principais de cada entidade no DynamoDB. Isso simplifica a busca por informações essenciais de um item, sendo uma 💡 **Dica de Ouro** para sua modelagem!

Exemplo 1: Autenticação

USER#zelda + SUMMARY → dados de autenticação

Exemplo 2: Operacional

EMPLOYEE#zelda + SUMMARY → dados operacionais

Exemplo 3: Dados do plano

PLAN#plano1 + SUMMARY → dados do plano

→ **Consistência:** Todos sabem onde buscar as informações primárias.

→ **Previsibilidade:** Um padrão claro e fácil de entender para a modelagem.

→ **Clareza no código:** A estrutura do SK se torna auto-documentada.

Convenção da Onze

Importante: Esta é nossa convenção interna, não uma regra imposta pelo DynamoDB. É uma prática recomendada para organização e eficiência!

5 Séries Temporais: Data no Sort Key

✗ ERRADO (como SQL)

```
PK: CERTIFICATE#cert1
SK: BALANCE
date: "2024-11-15"
```

Problema: Como buscar o último? Scan caro e lento!

✓ CERTO (DynamoDB way)

```
PK: CERTIFICATE#cert1
SK: BALANCE#2024-11-15
```

Mágica: Query reversa = último em 3ms!

🧠 Por que funciona:

1. DynamoDB ordena SK **lexicograficamente**
2. Data ISO 8601: "2024-11-01" < "2024-11-30"
3. Query + ScanIndexForward=false + Limit=1 = **último item instantâneo**

✂ Exemplo prático:

Certificado com 365 saldos diários

SQL:

```
SELECT * FROM balances ORDER BY date DESC LIMIT 1
```

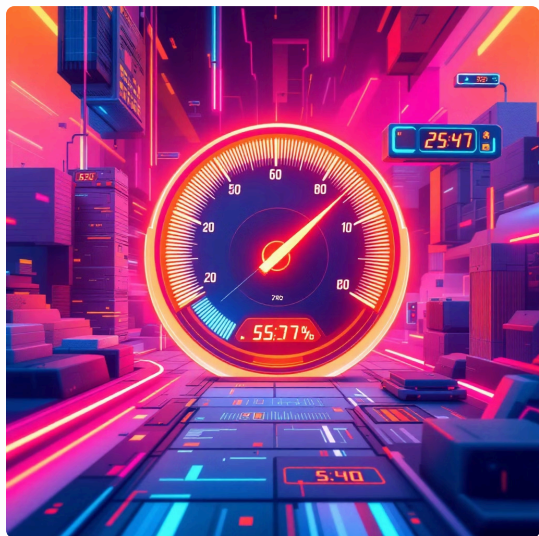
→ Scan + Sort (lento)

DynamoDB: Query reversa → 3ms, sempre!

💎 Bônus: O Poder do SK Ordenado

O segredo para a alta performance em séries temporais no DynamoDB é a propriedade de ordenação do Sort Key. Ao armazenar a data como parte do SK, você permite que o DynamoDB utilize suas capacidades de indexação eficientes para buscar rapidamente o item mais recente ou um intervalo de itens.

6 Denormalização: Duplicar para Velocidade



Quest: SQL Normalizado ✂

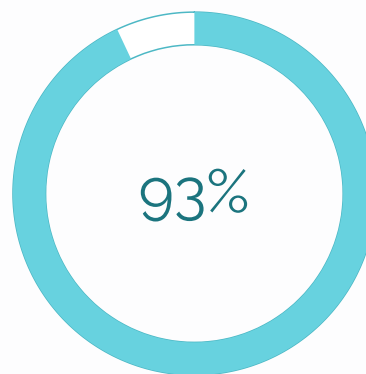
EMPLOYEE → user_id → JOIN tb_user

Recompensa: DynamoDB Denormalizado 💎

```
PK: EMPLOYEE#zelda
SK: SUMMARY
-> Retorno:
{
  employee_uuid: zelda,
  salary: 5000,
  user: {
    name: 'zelda'
    cpf: '123456789012'
    ...
  }
  ...
}
```

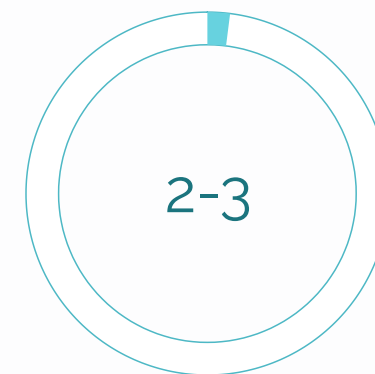
1 query, zero JOINS, 3ms (rápido)

🧠 Conhecimento: O Grande Trade-off



Mais rápida

500.000 leituras/mês



Updates extras

Quando nome muda (2x/mês)

Ratio: 250.000 : 1 → Vale MUITO a pena!

📋 Regra de Ouro da Denormalização:

1. Dados mudam **raramente**
2. São muito acessados **juntos**
3. **Leituras >> Escritas**

7 Filter Expression: Refinando Resultados da Query

O **Filter Expression** é uma ferramenta poderosa no DynamoDB. Ele aplica um filtro **DEPOIS** que os dados são lidos, refinando o resultado diretamente no servidor antes de serem enviados para sua aplicação.

🧠 Como funciona:

1. A **Query** busca itens com base na Partition Key (PK) e, opcionalmente, na Sort Key (SK).
2. O DynamoDB lê todos os itens que correspondem à sua PK e SK (por exemplo, 10 itens).
3. O **Filter Expression** é aplicado em todos esses dados lidos.
4. Apenas os itens que atendem aos critérios do filtro são retornados (por exemplo, 3 itens).

✂️ Exemplo Prático: Filtrando Planos Ativos

Imagine que você quer buscar apenas os planos ATIVOS de um funcionário:

Query Sem Filter

```
Query: PK="EMPLOYEE#zelda" AND begins_with(SK, "PLAN#")
```

➡️ Retorna TODOS os planos (ativos, inativos, cancelados) de Alice.

Query COM Filter

```
Query: PK="EMPLOYEE#zelda" AND begins_with(SK, "PLAN#")
```

```
Filter: status = "active"
```

➡️ Retorna APENAS os planos de Alice que estão com `status = "active"`.

💎 Tipos de Filtros Disponíveis:

- **Igualdade:** `status = "active"`
- **Comparação:** `salary > 5000`
- **Existência:** `attribute_exists(email)`
- **Contém:** `contains(tags, "premium")`
- **Entre valores:** `balance BETWEEN 1000 AND 5000`

🏛️ Importante:

- Filter NÃO reduz os **dados lidos** (Read Capacity Units), apenas os dados RETORNADOS.
- É útil para refinar resultados sem a necessidade de lógica de filtragem no seu código.

💡 **Aha moment:** A Query decide o que **BUSCAR**, o Filter decide o que **RETORNAR**!

Recap: 7 Conceitos Essenciais



1. Mudança de mentalidade

Como usar define como modelar



2. Access Pattern: A Pergunta que Define Tudo

Como criar padrões de acesso



3. Prefixos como Namespace

Namespace auto-documentado



4. Padrão SUMMARY

Convenção clara e consistente



5. Séries Temporais

Data no SK = ordenação grátis



6. Denormalização

Duplicar para velocidade



7. Filter expression

Refinando resultados da query

Agora você tem o arsenal completo para modelar! 🏰

🔪 Hora da Prática: Sua Primeira Missão!

🧑 Agora vamos aplicar esses conceitos e desvendar novos conhecimentos!

Access Patterns Reais

Sua lista de objetivos para esta missão:

- Buscar dados do employee
- Listar todos os planos de um employee
- Buscar último saldo de um certificado
- Listar certificados de um plano

Desafio de Modelagem 🧠

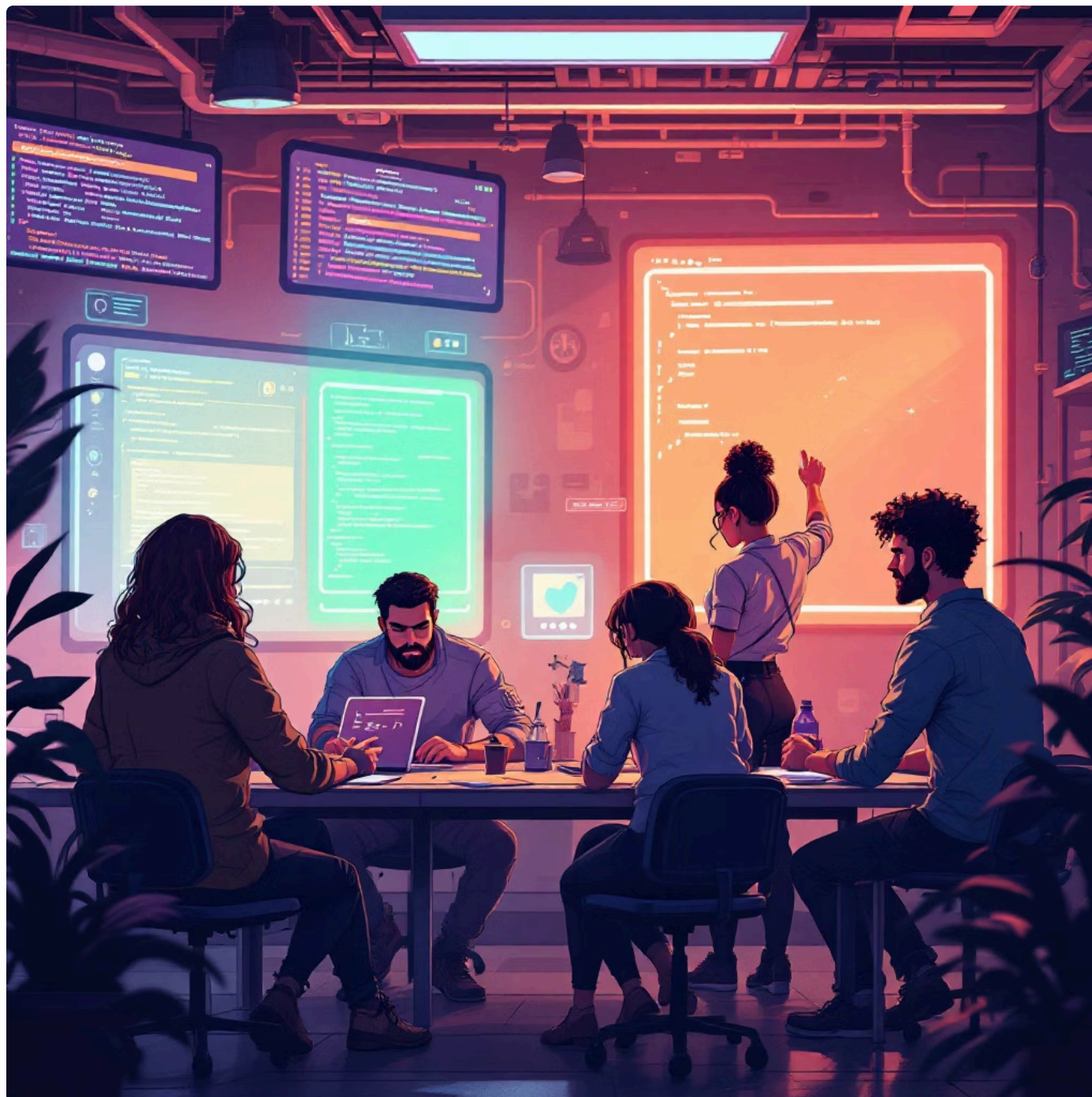
Modelar PK/SK para resolver cada um desses padrões de acesso, garantindo máxima performance no DynamoDB.

Dinâmica da Missão 🤝

Como vamos trabalhar juntos:

- Construímos a estrutura final juntos, passo a passo!

Vamos nessa! 💪





Você está aqui: Mão na Massa!

Quest 2 de 4

Onde estamos:

-  Modelagem Teórica



Modelagem Prática (20min)

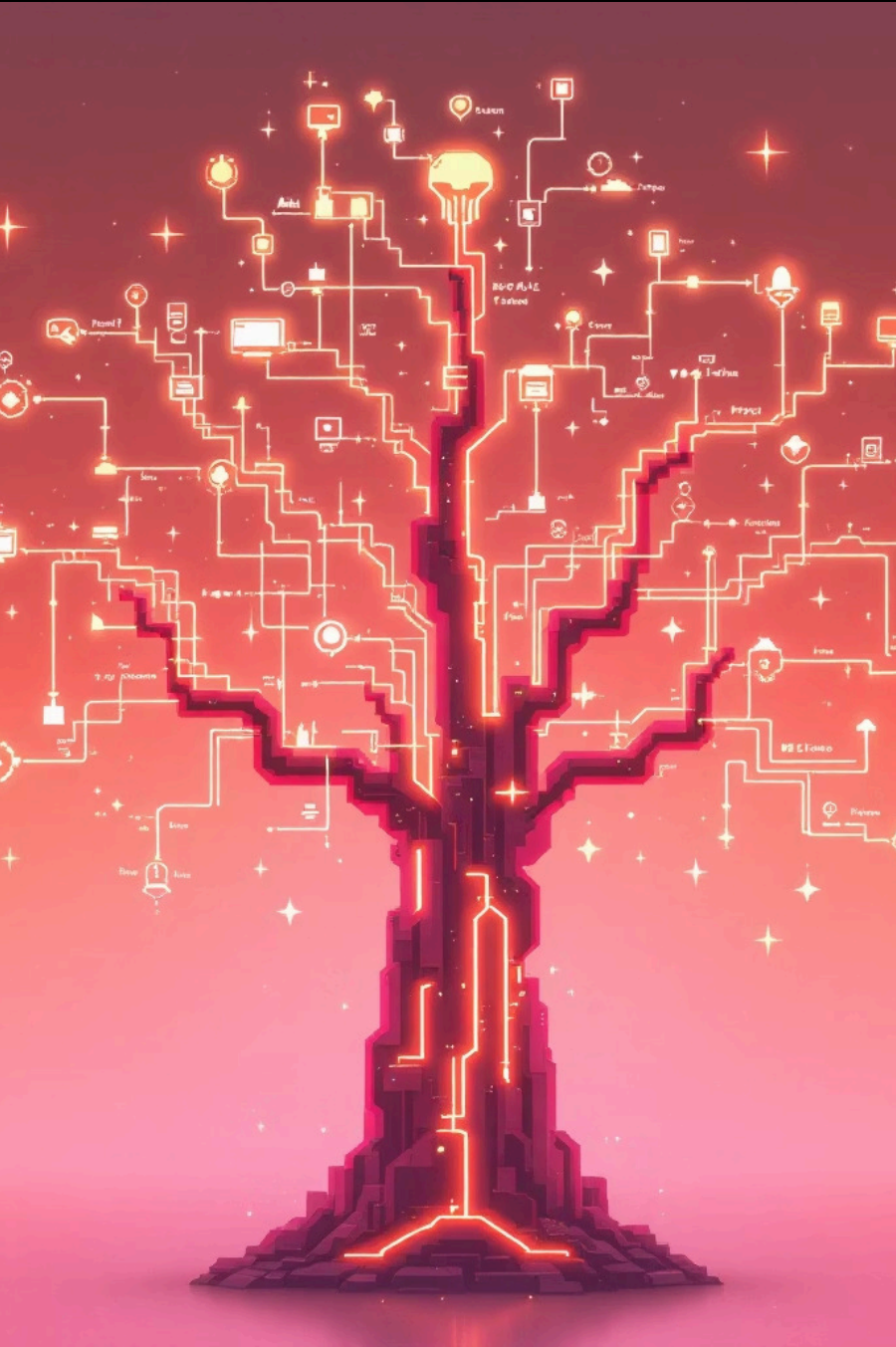
- |— Cenário real apresentado
- |— Modelar PK/SK em grupo
- |— Discussão das soluções

Próximos passos:

-  Arena: Queries básicas
-  Quiz 2: Access Patterns + Queries



Dica: Não existe resposta única! Discutam trade-offs.



Você está aqui: Arena: Queries básicas

Quest 2 de 4

Onde estamos:

-  Modelagem Teórica
-  Modelagem Prática

Próximo:

-  Quiz 2: Access Patterns + Queries



AGORA: Arena:
Queries básicas

Você está aqui: Quiz 2

Quest 2 de 4

Onde estamos:

-  Modelagem Teórica
-  Modelagem Prática
-  Arena: Queries básicas

 AGORA: Quiz 2

Recompensas:

-  XP disponível: 600 pontos
-  Badge:  Access Pattern Architect

Próximo:

 PAUSA MERECIDA!

